

Brute-Force String Matching

1 Introduction

String matching is an important problem in computer science.

The basic problem is:

Given a text and a pattern, find whether the pattern occurs inside the text.

For example:

Text = ABABCAB

and

Pattern = ABC

The pattern occurs in the text starting at index 2.

Therefore, the answer is:

2

String matching is used in many applications such as:

- Searching for a word in a document.
- Finding keywords in text.
- Searching in web pages.
- Finding patterns in DNA sequences.
- Searching text in software applications.

2 String Matching Problem

Let:

$T[0 \dots n - 1]$

represent the **text** of length n .

Let:

$P[0 \dots m - 1]$

represent the **pattern** of length m .

Usually:

$$m \leq n$$

The objective is to find the first position i such that:

$$T[i] = P[0]$$

$$T[i + 1] = P[1]$$

$$T[i + 2] = P[2]$$

and so on until:

$$T[i + m - 1] = P[m - 1]$$

If a complete match is found, the algorithm returns the starting index i .

If no match is found, it returns:

$$\boxed{-1}$$

3 Important Terms

Term	Meaning
Text	The complete string in which we search
Pattern	The string that we want to find
n	Length of the text
m	Length of the pattern
i	Starting position in the text
j	Position within the pattern
Match	Corresponding characters are equal
Mismatch	Corresponding characters are different

4 Basic Idea of Brute-Force String Matching

The brute-force method is very simple.

It follows these steps:

1. Align the pattern with the beginning of the text.
2. Compare the characters from left to right.
3. If all pattern characters match, return the starting index.
4. If a mismatch occurs, shift the pattern one position to the right.
5. Start comparing again from the first character of the pattern.
6. Continue until the pattern can no longer fit inside the text.

The important point is:

$$\boxed{\text{After a mismatch, start again from } P[0]}$$

No information from the previous comparison is used.

This is why it is called the **brute-force** method.

5 Example

Consider:

$$T = \text{ABABCAB}$$

and:

$$P = \text{ABC}$$

Here:

$$n = 7$$

and:

$$m = 3$$

We start by aligning the pattern with the beginning of the text.

$$\begin{array}{cccccc} A & B & A & B & C & A & B \\ A & B & C & & & & \end{array}$$

Compare from left to right:

$$A = A \quad \checkmark$$

$$B = B \quad \checkmark$$

$$A \neq C \quad \times$$

There is a mismatch.

Therefore, shift the pattern one position to the right.

6 Tracing the Example

Text:

$$\text{A B A B C A B}$$

Pattern:

$$\text{A B C}$$

Alignment 1

$$\begin{array}{cccccc} A & B & A & B & C & A & B \\ A & B & C & & & & \end{array}$$

Comparisons:

$$A = A \quad \checkmark$$

$$B = B \quad \checkmark$$

$$A \neq C \quad \times$$

Comparisons:

3

Shift pattern by one position.

Alignment 2

$$\begin{array}{cccccc} A & B & A & B & C & A & B \\ & A & B & C & & & \end{array}$$

First comparison:

$$A \neq B$$

Mismatch occurs immediately.

Comparisons:

1

Shift the pattern again.

Alignment 3

$$\begin{array}{cccccc} A & B & A & B & C & A & B \\ & & A & B & C & & \end{array}$$

Compare:

$$A = A \quad \checkmark$$

$$B = B \quad \checkmark$$

$$C = C \quad \checkmark$$

All characters match.

Therefore:

Pattern found at index 2

Total comparisons:

$$3 + 1 + 3 = 7$$

7 Visual Trace

The matching process can be shown as:

```

A B A B C A B
A B C

```

Mismatch at third character.

Shift by one:

```

A B A B C A B
  A B C

```

Mismatch at first character.

Shift again:

```

A B A B C A B
    A B C

```

Complete match.

Pattern found at index 2

8 Why Does the Algorithm Stop at $n - m$?

Suppose:

$$n = 7$$

and:

$$m = 3$$

The pattern contains 3 characters.

The last possible starting position is:

$$n - m$$

Therefore:

$$7 - 3 = 4$$

Possible starting positions are:

$$0, 1, 2, 3, 4$$

At position 5, only two text characters remain:

$$T[5], T[6]$$

but the pattern needs three characters.

Therefore, position 5 cannot be a valid starting position.

Hence:

$$i = 0 \text{ to } n - m$$

9 Brute-Force String Matching Algorithm

```

1 BruteForceStringMatch(T, P, n, m)
2
3 for i = 0 to n-m
4     j = 0
5
6     while j < m and T[i+j] == P[j]
7         j = j + 1
8
9     if j == m
10        return i
11
12 return -1

```

The algorithm compares:

$$T[i + j]$$

with:

$$P[j]$$

If they are equal, j is increased.

If they are different, the current alignment fails and the pattern is shifted to the next position.

10 Time Complexity

The basic operation is the character comparison:

$$T[i + j] == P[j]$$

There are two loops.

The outer loop tries different starting positions of the pattern.

$$i = 0 \text{ to } n - m$$

Using:

$$u - l + 1$$

we get:

$$(n - m) - 0 + 1$$

Therefore:

$$\boxed{n - m + 1}$$

possible alignments exist.

11 Best Case

In the best case, the first character of the pattern does not match the corresponding text character at each unsuccessful alignment.

Therefore, only one comparison is required for each alignment.

Number of alignments:

$$n - m + 1$$

Therefore:

$$C(n, m) = n - m + 1$$

Hence:

$$\boxed{O(n)}$$

for fixed or bounded pattern length.

12 Worst Case

The worst case occurs when many characters match before a mismatch occurs, or when the pattern occurs near the end or does not occur.

For each alignment, up to m characters may be compared.

Number of alignments:

$$n - m + 1$$

Maximum comparisons:

$$C(n, m) = m(n - m + 1)$$

Therefore:

$$C(n, m) = m(n - m + 1)$$

Ignoring lower-order terms:

$$C(n, m) = O(nm)$$

Hence:

$$\boxed{\text{Worst-case time complexity} = O(nm)}$$

If:

$$m \approx n$$

then:

$$O(nm) = O(n^2)$$

Therefore, brute-force string matching can have:

$$\boxed{O(n^2)}$$

worst-case complexity.

13 Number of Comparisons

For each alignment, the maximum number of character comparisons is m .

The number of possible alignments is:

$$n - m + 1$$

Therefore:

$$\boxed{\text{Maximum comparisons} = m(n - m + 1)}$$

For example, if:

$$n = 10, \quad m = 4$$

then the number of alignments is:

$$n - m + 1$$

$$= 10 - 4 + 1$$

$$= 7$$

Maximum comparisons:

$$m(n - m + 1)$$

$$= 4(7)$$

$$\boxed{28}$$

14 Important Difference: Alignment and Comparison

Students should not confuse these two terms.

Alignment means placing the pattern at a particular starting position in the text.

For:

$$n = 7, \quad m = 3$$

number of alignments:

$$n - m + 1 = 5$$

These are:

$$i = 0, 1, 2, 3, 4$$

However, each alignment may require several character comparisons.

Therefore:

$$\boxed{\text{Alignments} = n - m + 1}$$

while:

$$\boxed{\text{Maximum comparisons} = m(n - m + 1)}$$

15 Successful and Unsuccessful Search

There are two possible outcomes.

Successful Search

The pattern is found in the text.

Example:

$$T = \text{ABABCAB}$$

$$P = \text{ABC}$$

The pattern is found at:

$$\boxed{2}$$

Therefore, the algorithm returns 2.

Unsuccessful Search

The pattern does not occur in the text.

Example:

$$T = \text{ABABCAB}$$

$$P = \text{XYZ}$$

The algorithm checks all possible positions.

Since no match is found:

$$\boxed{-1}$$

is returned.

16 Advantages

- Very simple to understand.
- Easy to implement.
- Does not require preprocessing of the pattern.
- Works on unsorted text.
- Requires very little extra memory.
- Suitable for small inputs.

17 Disadvantages

- It may perform the same comparisons repeatedly.
- It does not use information from previous mismatches.
- It can be slow for large texts and patterns.
- Worst-case time complexity can be $O(nm)$.

18 Quick Revision

Concept	Result
Text length	n
Pattern length	m
Possible starting positions	$n - m + 1$
Starting index	0 to $n - m$
Basic operation	Character comparison
Best case	$O(n)$
Worst case	$O(nm)$
If $m \approx n$	$O(n^2)$
Extra memory	$O(1)$
Successful result	Starting index
Unsuccessful result	-1

19 Key Points to Remember

1. Text has length n .
2. Pattern has length m .
3. The pattern is shifted one position at a time.
4. Characters are compared from left to right.
5. When a mismatch occurs, comparison starts again with the first character of the pattern.
6. The last possible starting position is:

$$n - m$$

7. Therefore, the number of possible alignments is:

$$n - m + 1$$

8. Maximum comparisons are:

$$m(n - m + 1)$$

9. Worst-case complexity is:

$$\boxed{O(nm)}$$

10. When m is of the same order as n :

$$\boxed{O(n^2)}$$